Application Deployment

(Deploy the given React application to a production ready state)

Application:

Clone the below mentioned repository and deploy the application (Run application in port 3000).

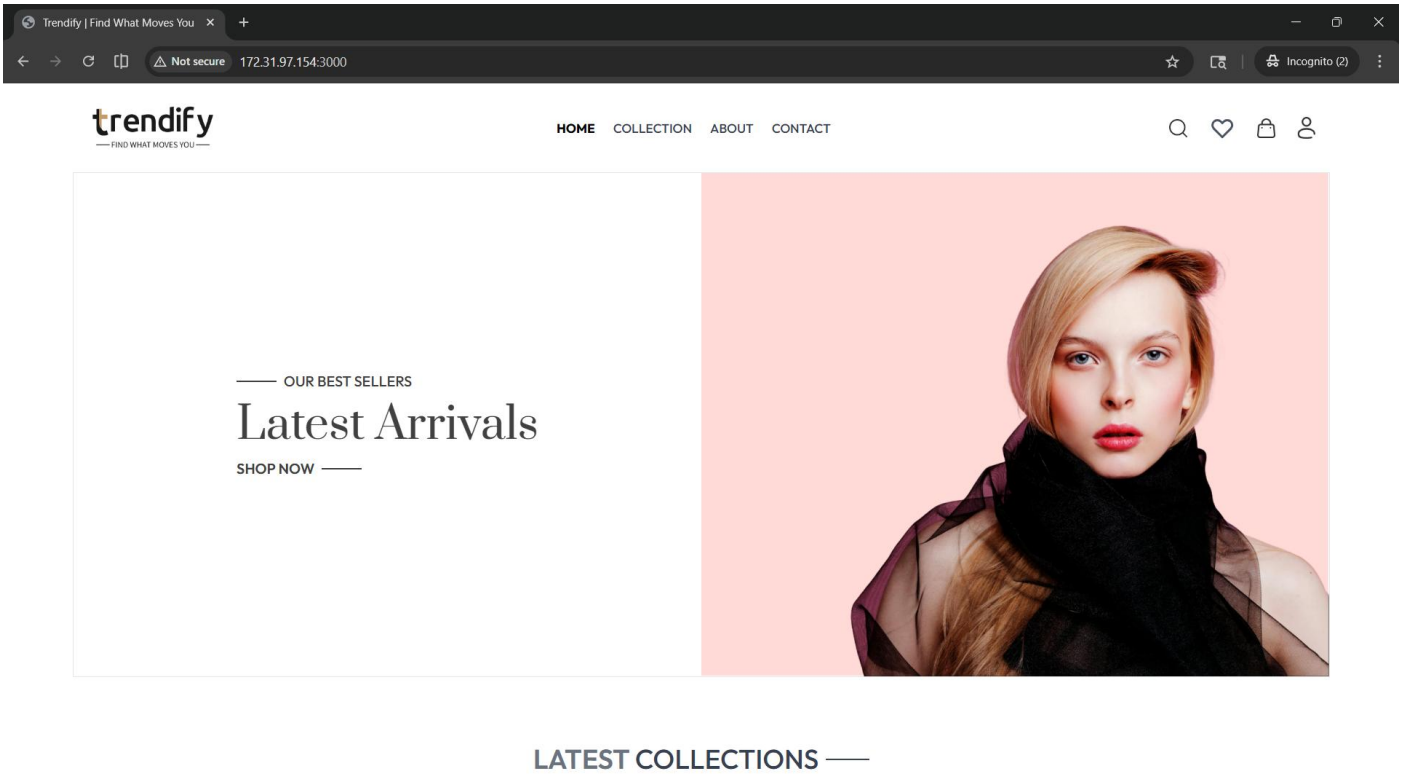
Repo URL : https://github.com/Vennilavanguvi/Trend.git

Docker:

- Dockerize the application by creating Dockerfile

```
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ docker images
REPOSITORY                    TAG       IMAGE ID       CREATED        SIZE
ghostatdocker/trendstore-image   latest    d00c47ee79e1   5 hours ago    71.4MB
trendstore-image              latest    d00c47ee79e1   5 hours ago    71.4MB
nginx                         alpine    5bd7bd52e5bc   6 days ago     62.2MB
hello-world                   latest    1b44b5a3e06a   8 months ago   10.1kB
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ docker run -d -p 3000:3000 --name trendstore-container trendstore-image
49fd4acaacd0d190a79ee464fcc784c585ca2e0a45c8e9ffdf93abfc9a0c2bba

GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ hostname -I
172.31.97.154 172.17.0.1 192.168.49.1
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$
```

- Build an application and check output using docker image.



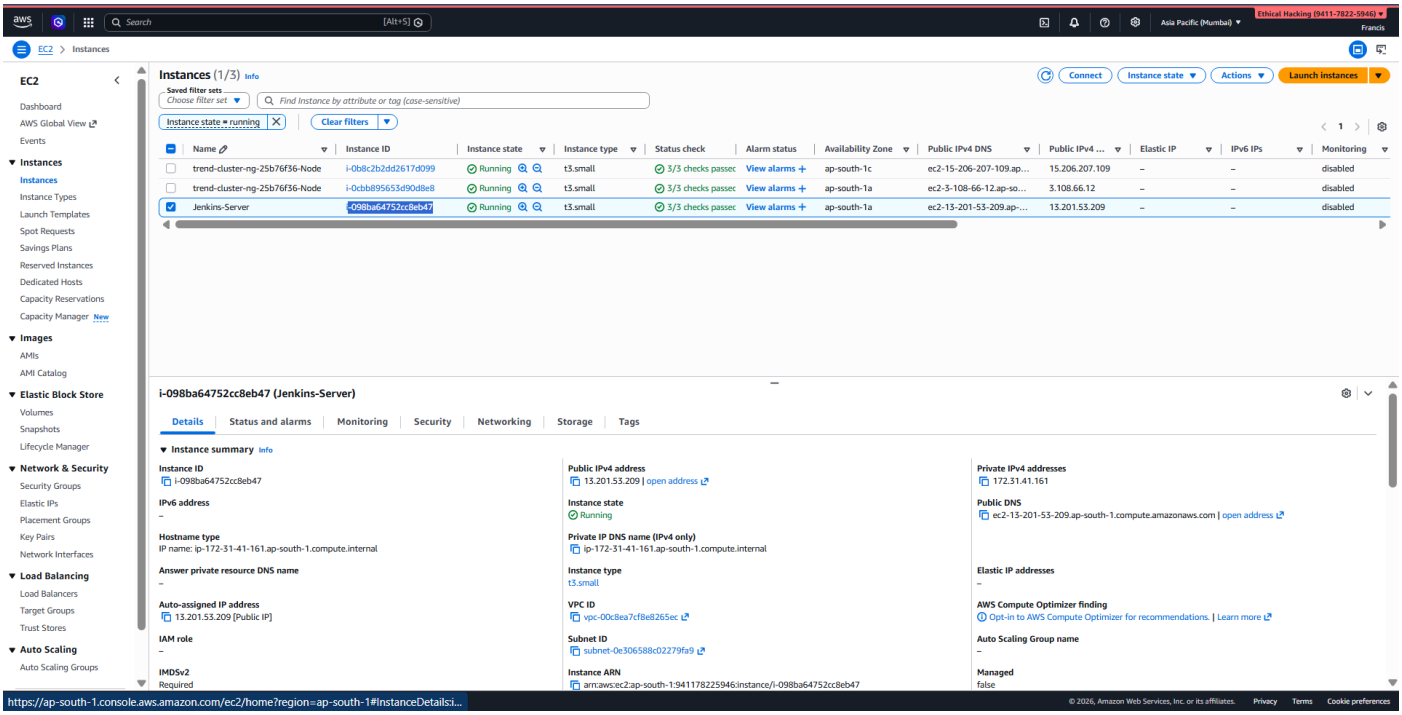
Terraform:

- Define infrastructure in main.tf to create VPC, IAM, EC2 with Jenkins, etc.

```
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ cat infra/main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

# ----------------------------
# SECURITY GROUP FOR JENKINS
# ----------------------------
resource "aws_security_group" "jenkins_sg" {
  name        = "jenkins-sg"
  description = "Allow Jenkins, SSH, HTTP"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Jenkins UI"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP (optional)"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "jenkins-sg"
  }
}
```

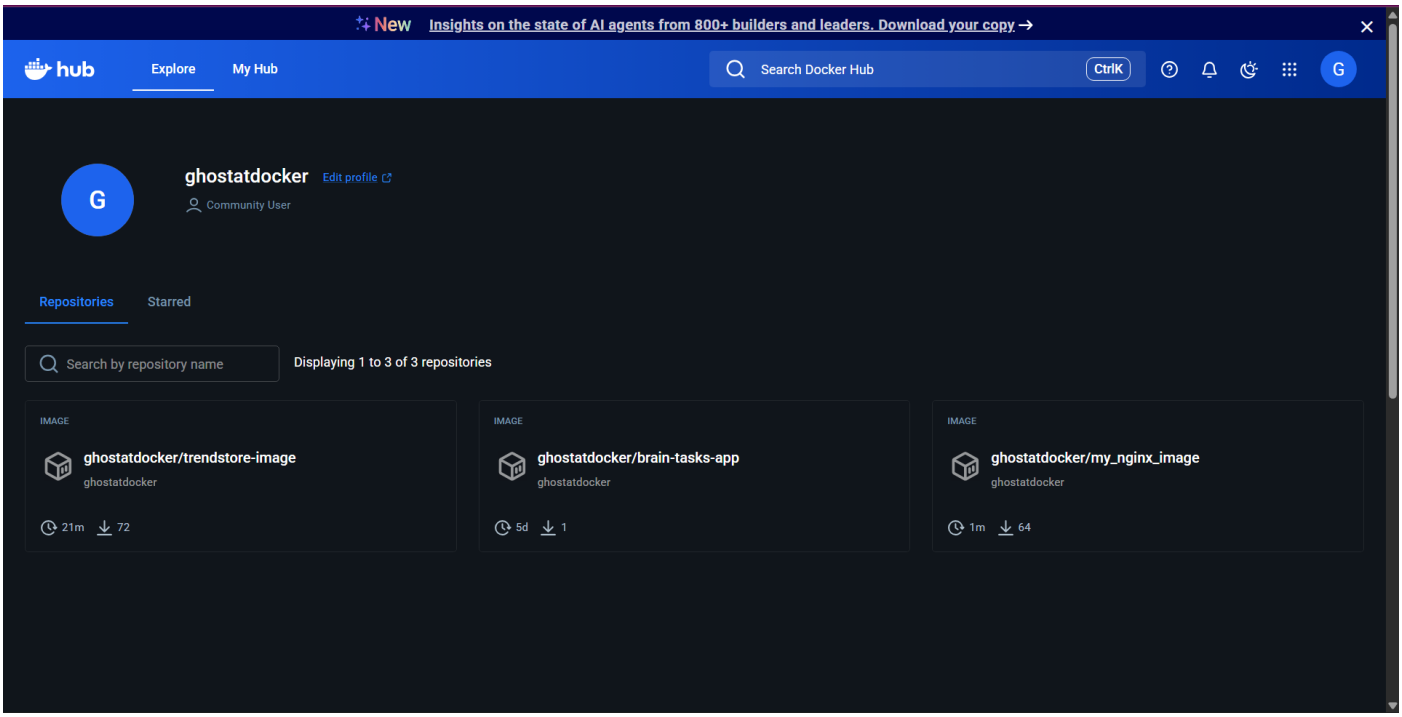
```
# ----------------------------
# EC2 INSTANCE (JENKINS SERVER)
# ----------------------------
resource "aws_instance" "jenkins" {
  ami           = "ami-0f5ee92e2d63afc18" # Amazon Linux 2 (ap-south-1)
  instance_type = "t3.micro"

  key_name      = "your-key-name" # CHANGE THIS

  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    yum update -y

    # Install Docker
    amazon-linux-extras install docker -y
    systemctl start docker
    systemctl enable docker
    usermod -aG docker ec2-user

    # Install Java (Jenkins dependency)
    amazon-linux-extras install java-openjdk11 -y

    # Install Jenkins
    wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
    rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io.key
    yum install jenkins -y

    systemctl enable jenkins
    systemctl start jenkins

    # Install Git
    yum install git -y
  EOF

  tags = {
    Name = "Jenkins-Server"
  }
}

# ----------------------------
# OUTPUT
# ----------------------------
output "jenkins_public_ip" {
  value = aws_instance.jenkins.public_ip
}
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$
```

- Use terraform command to provision infrastructure.



DockerHub:

- Create a DockerHub repository.



Kubernetes:

- Setup Kubernetes in AWS EKS and Confirm EKS cluster is running.

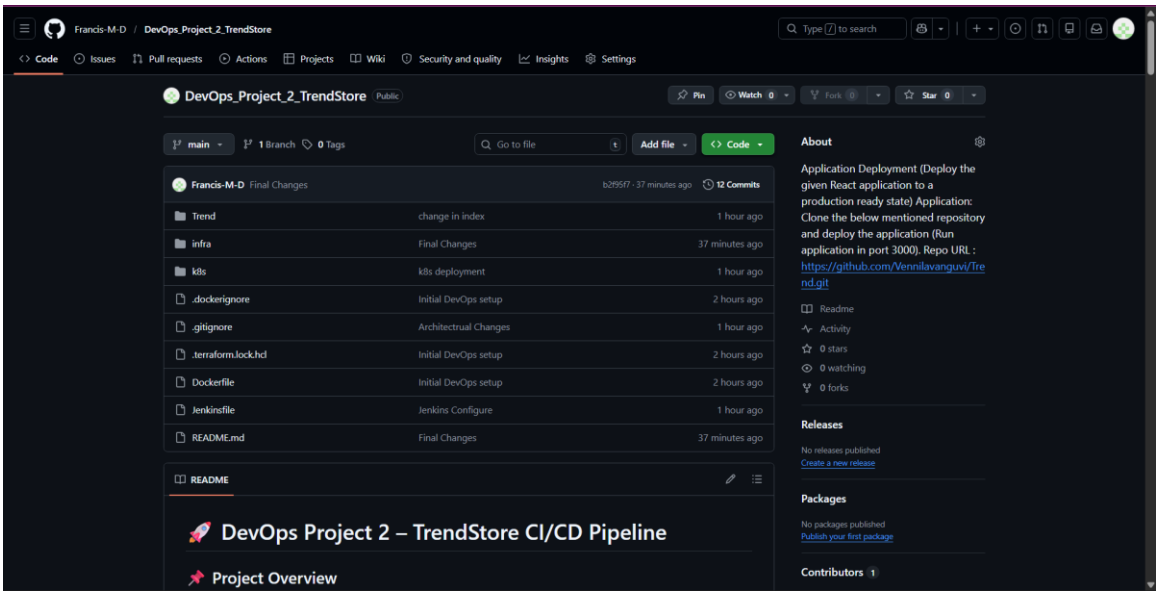
```
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ aws eks list-clusters
{
    "clusters": [
        "trend-cluster"
    ]
}
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ kubectl get nodes
NAME                                          STATUS   ROLES    AGE    VERSION
ip-192-168-11-201.ap-south-1.compute.internal   Ready    <none>   179m   v1.34.4-eks-f69f56f
ip-192-168-64-253.ap-south-1.compute.internal   Ready    <none>   179m   v1.34.4-eks-f69f56f
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ kubectl get pods -A
NAMESPACE     NAME                                                     READY   STATUS    RESTARTS   AGE
default       alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   0          58m
default       monitoring-grafana-76979794b5-m4p89                      3/3     Running   0          44m
default       monitoring-kube-prometheus-operator-bb98b8cfc-7gbmg      1/1     Running   0          58m
default       monitoring-kube-state-metrics-5b8ff7679f-6tfqs           1/1     Running   0          58m
default       monitoring-prometheus-node-exporter-5rx2x                1/1     Running   0          58m
default       monitoring-prometheus-node-exporter-94tc7                1/1     Running   0          58m
default       prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   0          58m
default       trend-store-app-7ccdc8f4-rf765                           1/1     Running   0          32m
default       trend-store-app-7ccdc8f4-tqqvd                           1/1     Running   0          32m
kube-system   aws-node-n8fpg                                           2/2     Running   0          179m
kube-system   aws-node-vb859                                           2/2     Running   0          179m
kube-system   coredns-c57cc66d8-8ss76                                  1/1     Running   0          3h3m
kube-system   coredns-c57cc66d8-mjvhc                                  1/1     Running   0          3h3m
kube-system   kube-proxy-7jtdn                                         1/1     Running   0          179m
kube-system   kube-proxy-f6nsn                                         1/1     Running   0          179m
kube-system   metrics-server-5c787d79d9-cgv8j                          1/1     Running   0          177m
kube-system   metrics-server-5c787d79d9-gpmpm                          1/1     Running   0          177m
```

- Write deployment and service YAML files.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-store-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: trend
  template:
    metadata:
      labels:
        app: trend
    spec:
      containers:
      - name: trendstore
        image: ghostatdocker/trendstore-image:latest
        imagePullPolicy: Always
~
~
~
```

Version Control:

- Push the codebase to a Git provider (GitHub).

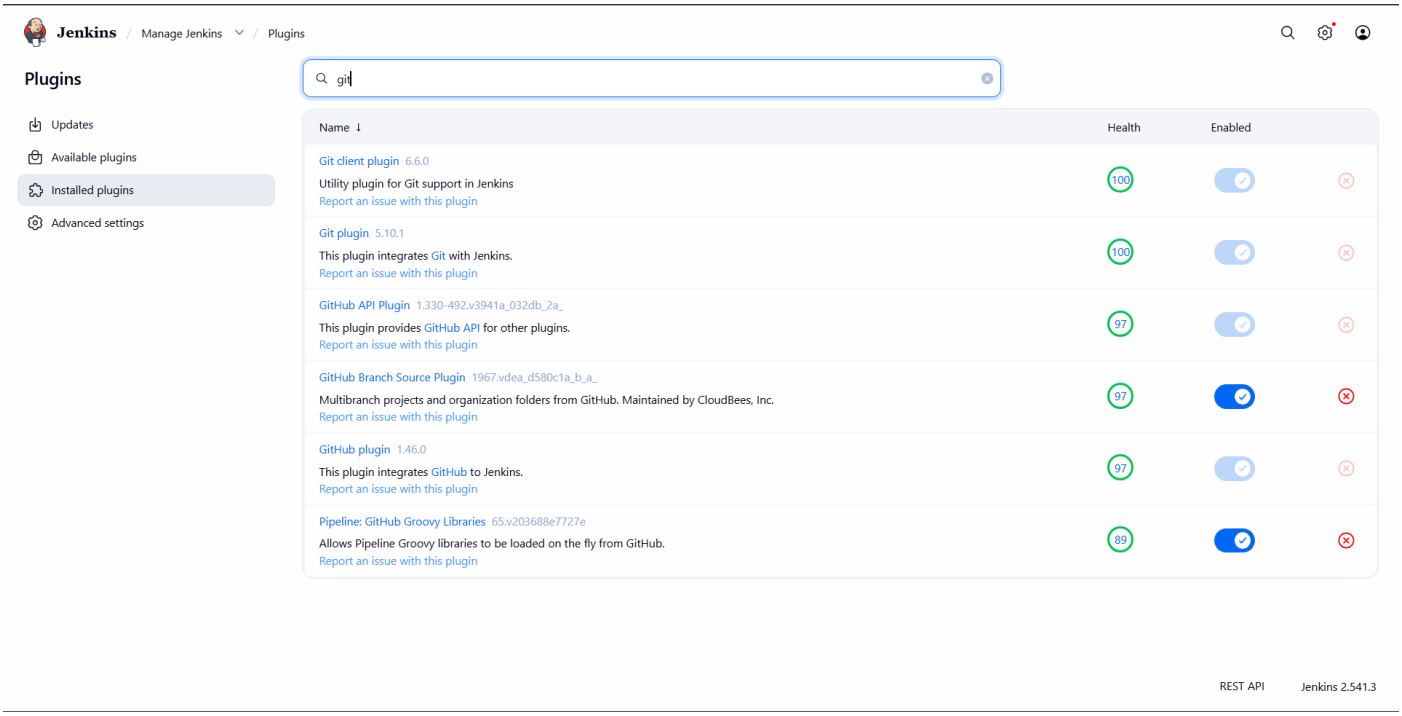


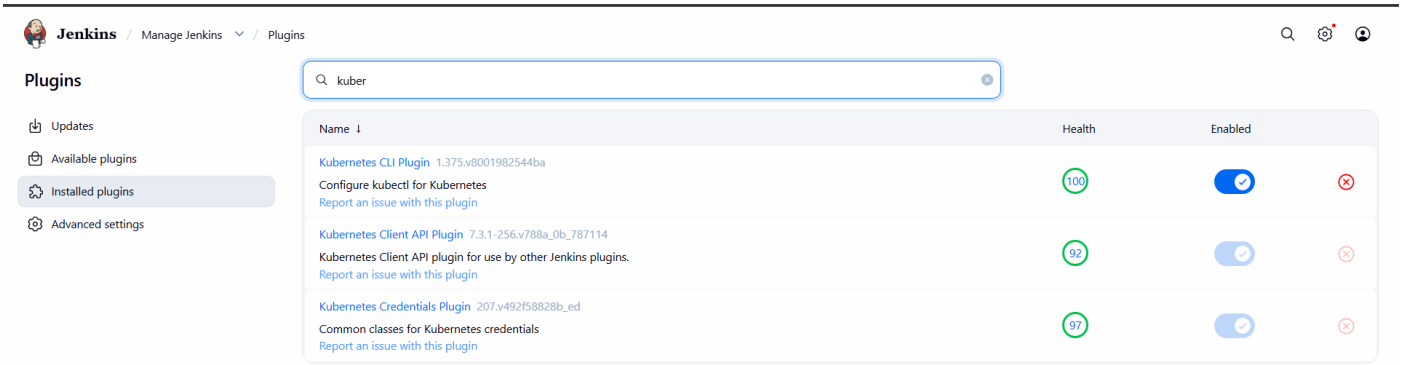
- Add gitignore and dockerignore files and use CLI commands to push code.

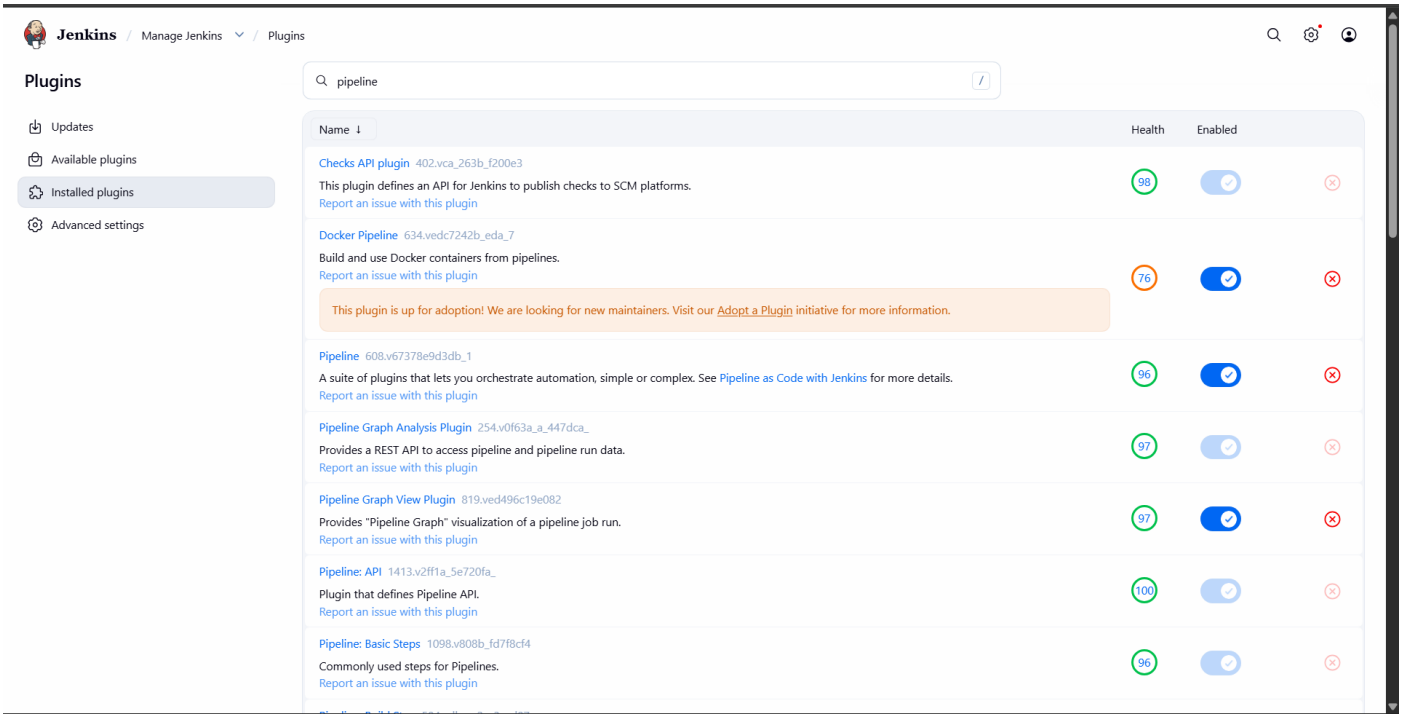
```
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ ls -la | grep -i "git"
drwxr-xr-x 8 GHOST GHOST 4096 Apr 14 12:22 .git
-rw-r--r-- 1 GHOST GHOST  246 Apr 14 13:02 .gitignore
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ cat .gitignore
.terraform/
*.tfstate
*.tfstate.*
crash.log
crash.*.log
*.tfvars
*.tfvars.json
override.tf
override.tf.json
*_override.tf
*_override.tf.json
.terraform.tfstate.lock.info
.terraformrc
terraform.rc
terraform.tfstate
terraform.tfstate.*
.terraform/
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$
```

Jenkins:

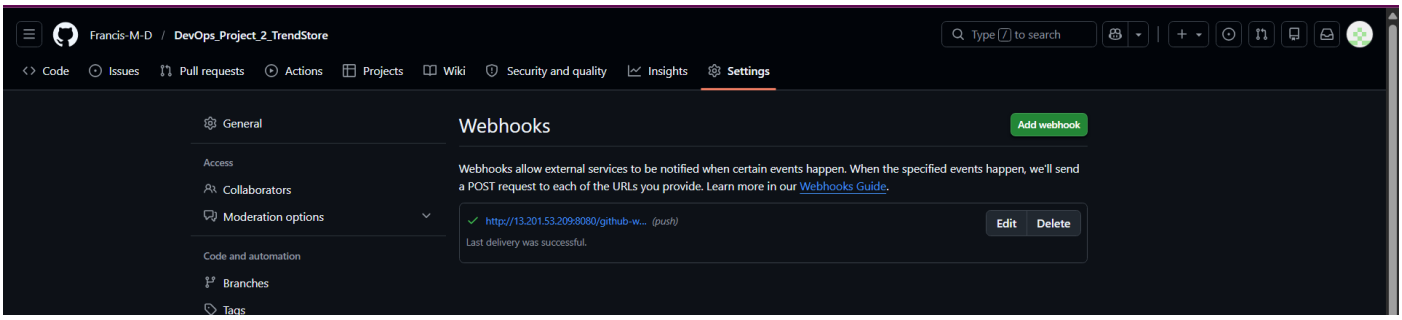
- Install Jenkins and necessary plugins (Docker, Git, Kubernetes, Pipeline) for build, push &deploy applications.

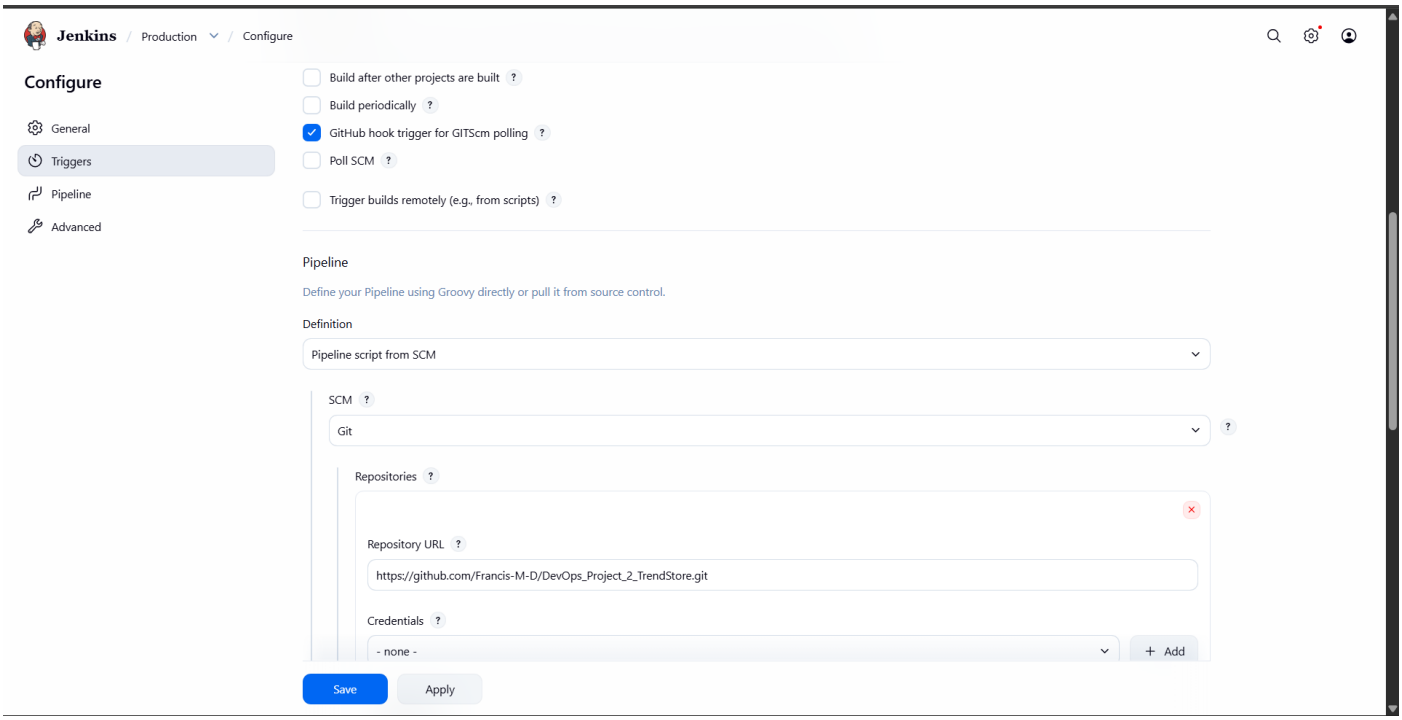






- Setup Github and jenkins integration using github webhook build trigger for auto build for every commit.



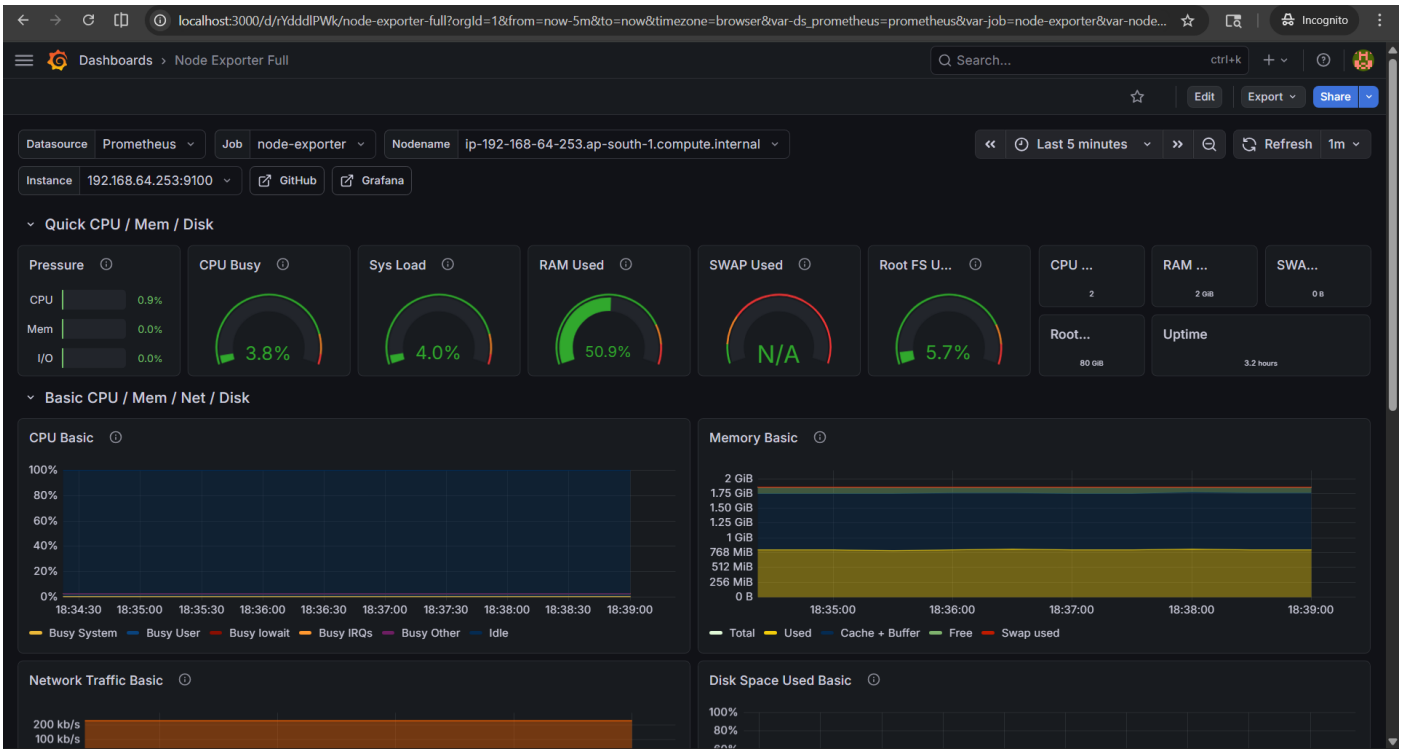


- Create a declarative pipeline script and pipeline project to build, push & deploy using CI-CD.

```
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ cat Jenkinsfile
pipeline {
    agent any

    environment {
                IMAGE = "ghostatdocker/trendstore-image"
                TAG = "${BUILD_NUMBER}"
        }

    stages {

                stage('Git Checkout') {
                        steps {
                                // Change 'main' to whatever your branch is named (e.g., 'master' or 'develop')
                                git branch: 'main', url: 'https://github.com/Francis-M-D/DevOps_Project_2_TrendStore.git'
                        }
                }

        stage('Docker Build') {
            steps {
                sh 'docker build -t $IMAGE:$TAG .'
            }
        }

        stage('Docker Push') {
            steps {
                // This securely injects your password from Jenkins credentials
                withCredentials([string(credentialsId: 'docker-hub-creds', variable: 'DOCKER_PASS')]) {
                    sh "echo \$DOCKER_PASS | docker login -u ghostatdocker --password-stdin"
                    sh "docker push $IMAGE:$TAG"
                }
            }
        }

        stage('Deploy to EKS') {
            steps {
                sh 'kubectl apply -f k8s/deployment.yaml'
                sh 'kubectl apply -f k8s/service.yaml'
            }
        }

                stage('Deploy to Kubernetes') {
                        steps {
                                sh '''
                                kubectl set image deployment/trend-store-app \
                                trendstore=ghostatdocker/trendstore-image:$TAG

                                kubectl rollout status deployment/trend-store-app
                                '''
                        }
                }
                stage('Verify Deployment') {
            steps {
                sh '''
                kubectl get pods
                kubectl get svc
                '''
            }
        }
        }
        post {
                success {
                        echo "✅ Pipeline executed successfully!"
                }
                failure {
                        echo "❌ Pipeline failed. Check logs."
                }
        }
}
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ |
```

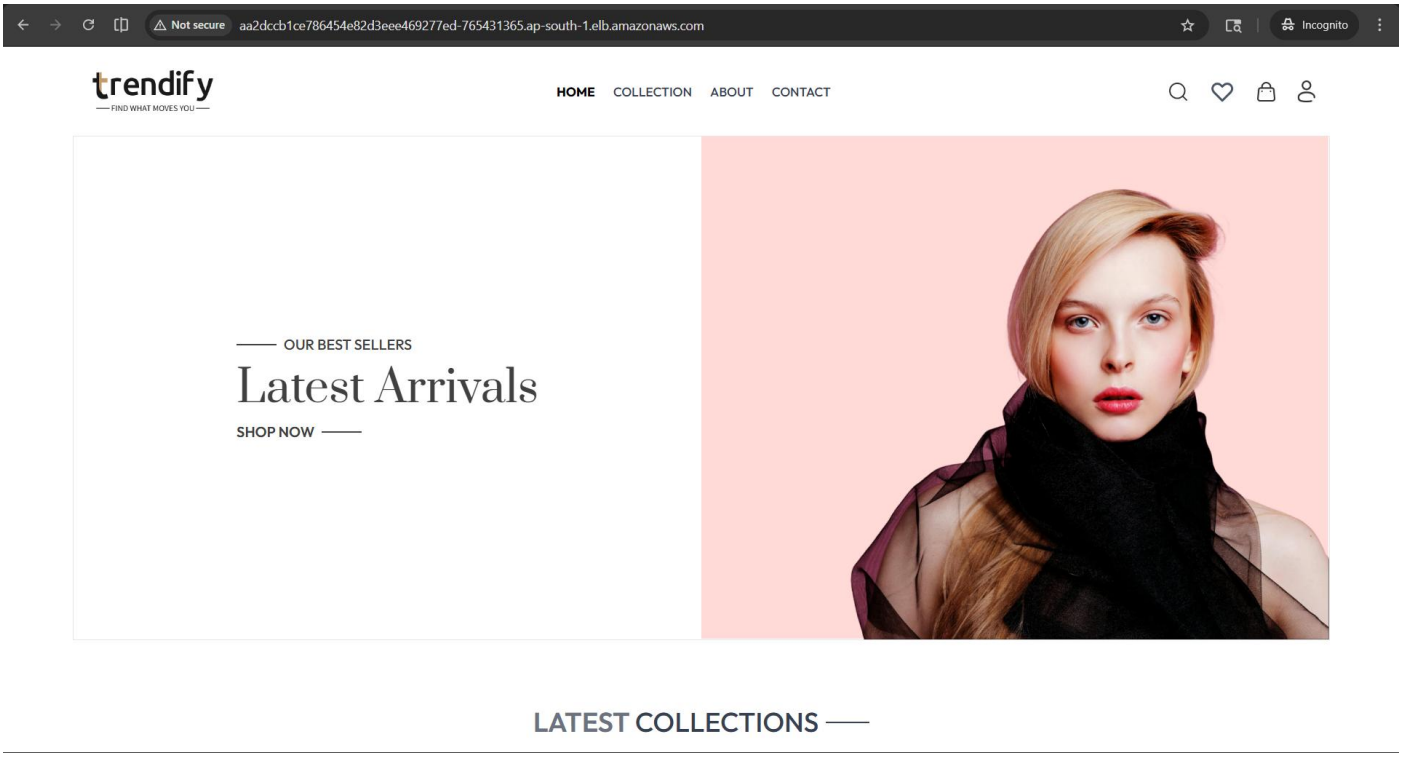
Monitoring:

- Setup a monitoring system to check the health of the cluster or application (opensource) is highly appreciable.

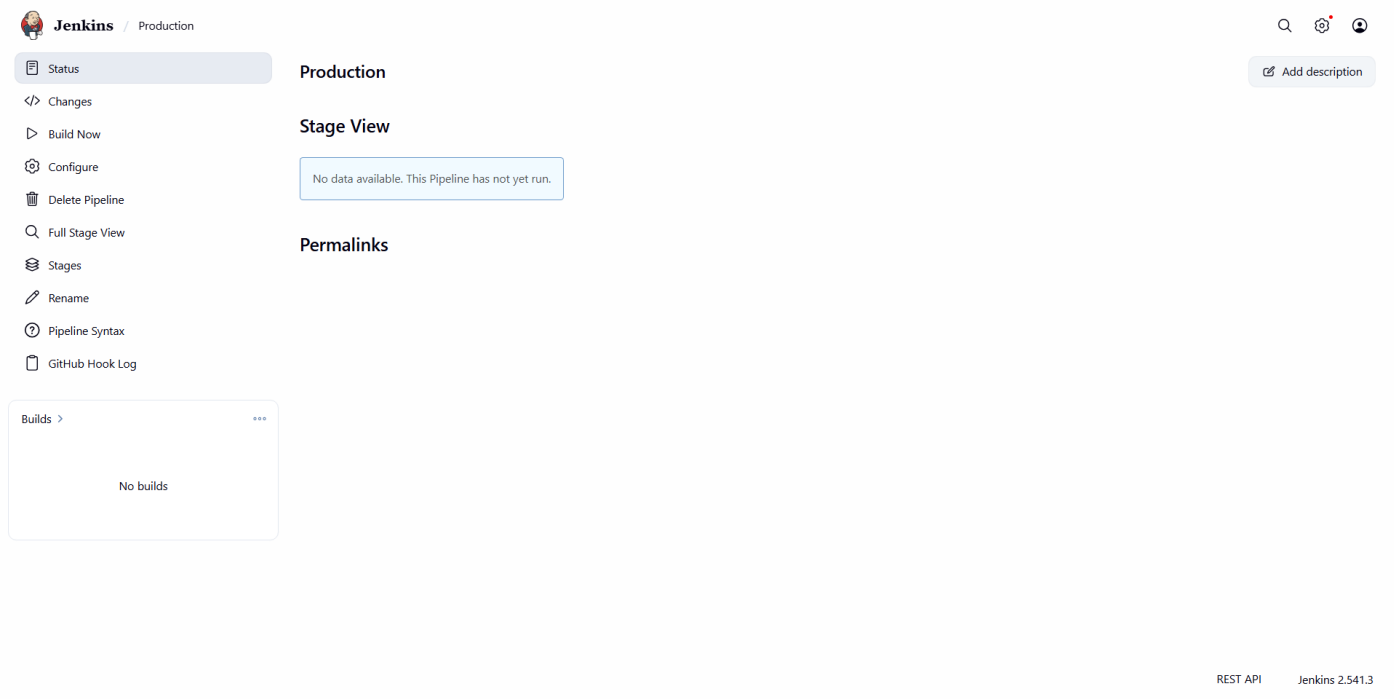


FINAL VERIFICATION:

Application Before Change:



Jenkins Before Pushing the code in Github

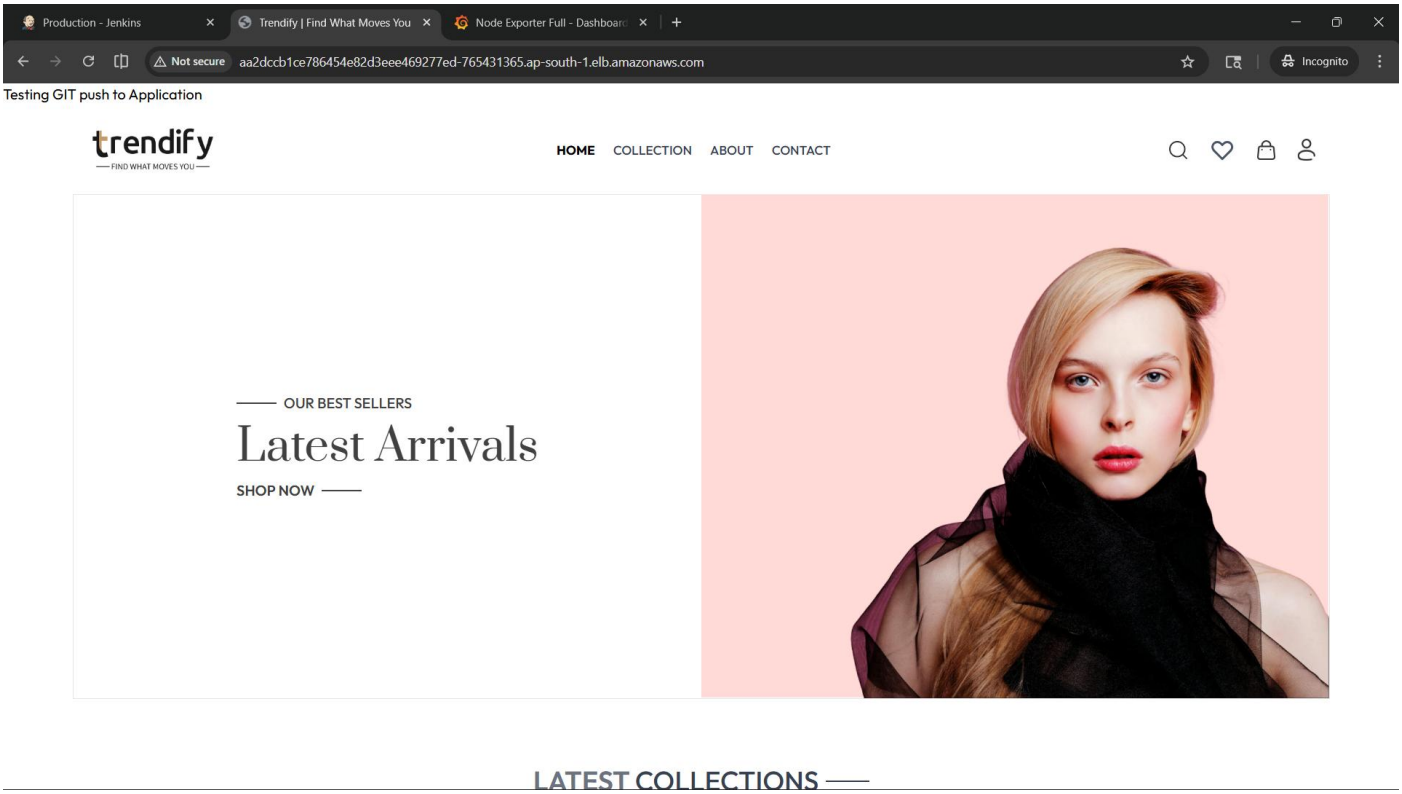


Pushing the code to github:

```
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ vi Trend/dist/index.html
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ cat Trend/dist/index.html | grep -i "h1"
        <h1>Testing GIT push to Application</h1>
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ git add .
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ git commit -m "Jenkins Pipeline Check"
[main f6940e9] Jenkins Pipeline Check
 3 files changed, 4 insertions(+), 24 deletions(-)
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$ git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 759 bytes | 253.00 KiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/Francis-M-D/DevOps_Project_2_TrendStore.git
   b2f95f7..f6940e9  main -> main
GHOST@GHOST:~/Final_Projects/DevOps_Project_2_TrendStore$
```

Jenkins triggered from Webhook after pushing the code to git:

Application after the change:



Submission Guidelines:

- GitHub Link: Submit full code repository.

  https://github.com/Francis-M-D/DevOps_Project_2_TrendStore

- README File: Include setup instructions, pipeline explanation, and screenshots or create a document and attach all screenshots.

- Application deployed kubernetes Loadbalancer ARN.

  http://aa2dccb1ce786454e82d3eee469277ed-765431365.ap-south-1.elb.amazonaws.com/